

가상화 기반 AI 솔루션

가상 데스크톱 인프라(VDI) 환경에서 사용자와 관리자가 AI를 통해 자원을 관리하고 장애 분석 및 상태를 진단할 수 있는 통합 솔루션입니다. Multi-Agent 구조의 AI Agent, vCenter 리소스 수집 엔진(vCompute), 이들을 연결하는 통합 API 서버 개발을 전담했습니다.

AI Agent (Python)

vCompute (C#)

API Server (Java)

Spring Boot

Spring AI

Spring Security

JWT

MCP Protocol

MSSQL

LangChain

OpenAI API

Claude API

Ollama

FastAPI

WebSocket

.NET Framework

NuGet

VMware Vim SDK

01

AI Agent — Python 기반 Multi-Agent 개발

PYTHON · LANGCHAIN · MULTI-AGENT ARCHITECTURE

핵심 목표

단일 에이전트의 Context Overload로 인한 환각(Hallucination) 현상을 구조적으로 방지하고, VDI 도메인에 특화된 사용자·관리자 지원 시나리오를 구현했습니다. LangChain 기반의 유연한 멀티 LLM 연동 환경도 함께 구축했습니다.

1-1. Multi-Agent 구조 설계

PROBLEM

Context Overload

단일 에이전트가 모든 VDI 도메인 지식·도구·히스토리를 하나의 Context에 담을 경우, 토큰 한계 초과 및 **환각(Hallucination) 현상**이 빈번하게 발생합니다.

SOLUTION

Main-Sub Agent 분리

Main Agent가 의도를 분류하고 Sub-Agent에 위임하는 **역할 분리 구조**로 설계했습니다. 각 Sub-Agent는 자신의 도메인에 필요한 최소한의 Context만 보유하므로, 응답의 정확도와 일관성이 크게 향상되었습니다.

STEP 01 · WEBSOCKET / FASTAPI

사용자 질문 수신 — 실시간 채팅 인터페이스

WebSocket 통신으로 실시간 응답을 제공합니다.

STEP 02 · MAIN AGENT

의도 분류 및 Sub-Agent 라우팅

질문 의도를 **사용자 지원** **관리자 지원** **일반 문의** 로 분류 후, 담당 Sub-Agent로 작업을 위임합니다.

STEP 03 · MEMORY MANAGER

대화 히스토리 기반 Context 유지

대화 이력(Conversation History)을 참조해 멀티턴(Multi-turn) 대화에서도 문맥이 끊기지 않도록 관리합니다.

STEP 04 · SUB-AGENTS (DOMAIN SPECIALIZED)

VDI 도메인 특화 작업 실행

사용자 지원 에이전트: VM 상태 이슈 자가 진단 및 조치 방법 안내, 사용자 자원 현황 조회.

관리자 지원 에이전트: 다수 사용자의 VDI 이력 추적, 상태 일괄 업데이트 등 반복 관리 업무 자동화.

STEP 05 · LANGCHAIN · LLM ROUTER

동적 LLM 선택 및 호출

프로젝트 특성·비용·성능 요구에 따라 아래 모델 중 최적 모델로 동적 교체가 가능합니다.

연동 LLM 목록 (LANGCHAIN 기반)

OpenAI GPT

Anthropic Claude

Gemini

Grok

Luxia

Friendli

Ollama (Local)

LangChain의 추상화 레이어를 활용해 모델 교체 시 비즈니스 로직 변경 없이 **config** 만으로 전환 가능하도록 구현했습니다.

1-2. VDI 도메인 특화 프롬프트 엔지니어링



정교한 도메인 지식

VMware VDI 환경(DataCenter, Cluster, Host, VM 계층 구조, 이벤트 유형)에 대한 도메인 지식을 시스템 프롬프트에 체계적으로 작성하여, 에이전트가 가상화 환경 특유의 이슈를 정확하게 해석하고 응답할 수 있도록 고도화했습니다.



사용자 지원 에이전트 시나리오

VM 상태 이슈(연결 불가, 성능 저하 등)를 AI가 자동 진단하고 **자가 조치 가이드**를 제공합니다. 사용자가 직접 자신의 VDI 자원 현황을 조회하고 관리할 수 있는 대화형 인터페이스를 구현했습니다.



관리자 지원 에이전트 시나리오

수십~수백 사용자의 VDI 이력을 추적하고, 상태 업데이트·이상 감지 등 **반복적인 관리 업무를 자동화**하는 시나리오를 설계했습니다. 관리자 생산성을 높이고 운영 오류를 줄이는 것이 핵심 목표입니다.



환각 방지 전략

역할 분리로 각 에이전트의 Context 범위를 최소화. 필요한 도구·지식만 주입해 응답 정확도를 높였습니다.



LLM 유연성

7종 이상 LLM을 `config` 변경만으로 교체 가능. 프로젝트 예산·성능 요건에 즉각 대응합니다.



자가 진단 UX

사용자가 IT 지식 없이도 VM 이슈를 AI와 대화하며 스스로 해결할 수 있는 경험을 제공합니다.



관리 자동화

반복적인 VDI 상태 추적·업데이트 업무를 에이전트가 자동 처리하여 관리자 운영 부담을 줄였습니다.

Python

FastAPI

LangChain

Multi-Agent

WebSocket

Prompt Engineering

Ollama

OpenAI

핵심 목표

.NET Framework 4.5.2와 VMware Vim SDK를 활용해 vCenter 계층 구조(DataCenter → Cluster → Host → VM)의 리소스 정보와 이벤트·태스크 이력을 조회하는 REST API를 개발했습니다. SDK의 페이징 부재로 인한 **서버 과부하·응답 지연·먹통 현상**을 **서버 측 Count 제한**과 **호출 모듈의 커서 기반 순차 재조회** 설계로 해결하여 대규모 인프라 환경에서도 안정적인 성능을 확보했습니다.

2-1. VMware Vim SDK 연동 및 REST API 개발



vCenter 이벤트·태스크 수집 API

.NET Framework 4.5.2 + VMware.Vim SDK를 활용해 DataCenter, Cluster, Host, VM의 리소스 현황과 **이벤트(Event)** 및 **태스크(Task)** 이력을 조회하는 REST API를 설계·개발했습니다. API는 `startTime` (시작시간)과 `id` (선택)를 파라미터로 수신합니다.

2-2. 커서 기반 순차 재조회 설계

PROBLEM

SDK 페이징 부재 · 대량 응답 문제

VMware Vim SDK에는 표준 Paging 기능이 없어, 한 번의 호출로 **수만 건이 반환**될 수 있습니다. 이를 제한 없이 허용하면 **서버 과부하 · 응답 지연 · 먹통 현상**이 발생합니다.

SOLUTION

서버 Count 제한 + 호출 모듈 순차 재조회

엔진 서버는 `web.config`의 **최대 조회 건수(Count)**만큼만 반환하고, 외부 **호출 모듈**이 반환된 마지막 이벤트/태스크의 시작시간과 ID를 커서로 삼아 다음 구간을 재조회하는 방식으로 전체 데이터를 순차 수집합니다.



API 파라미터 분기 — `startTime` + `id` (선택)

호출 모듈은 두 가지 파라미터를 엔진 API에 전달합니다.

① **id 없이 호출** — `startTime`부터의 이벤트·태스크를 Count 건만큼 반환합니다. 최초 조회 또는 새 구간 시작 시 사용합니다.

② **id 포함 호출** — `startTime` 범위 내에서 해당 `id` 다음 레코드부터 조회합니다. 동일 타임스탬프 구간 내 누락 없이 이어서 수집할 때 사용합니다.

STEP 01 · 호출 모듈 → 최초 API 요청

startTime 전달, id 없이 첫 조회

호출 모듈이 수집 시작 시간(`startTime`)만 전달합니다. 엔진은 해당 시점부터 `web.config`의 `Count`만큼 이벤트·태스크를 조회하여 반환합니다.

STEP 02 · 엔진 → COUNT 단위 반환

최대 조회 건수만큼 이벤트·태스크 응답

서버는 Count 건을 채워 응답합니다. 응답에는 마지막 레코드의 **이벤트/태스크 시작시간**과 **고유 ID**가 포함됩니다.

STEP 03 · 호출 모듈 → 잔여 데이터 판단

반환 건수 == Count 이면 추가 데이터 존재로 판단

반환 건수가 `Count`와 같으면 호출 모듈은 잔여 데이터가 있다고 간주하고 재조회를 준비합니다. 반환 건수 < Count이면 수집 완료로 판단하고 종료합니다.

STEP 04 · 호출 모듈 → 커서로 재조회

마지막 시작시간 + id를 커서로 API 재호출

호출 모듈이 직전 응답의 마지막 **이벤트/태스크 시작시간**과 **id**를 파라미터로 엔진 API를 재호출합니다. 엔진은 해당 id 다음 레코드부터 Count만큼 반환하여 중복·누락 없는 연속 수집을 보장합니다.

STEP 05 · 반복 → 수집 완료

반환 건수 < Count 될 때까지 Step 02-04 반복

호출 모듈이 Step 02-04를 반복하며 전체 이벤트·태스크를 순차 수집합니다. 마지막 페이지에서 반환 건수가 Count보다 적으면 수집을 종료합니다.

2-3. 세션 버퍼 기반 메모리 캐싱 전략



원격 호출 최소화 → 즉시 메모리 반환

vCenter API의 네트워크 지연(RTT)과 서버 부하를 줄이기 위해 조회된 데이터를 세션 버퍼에 캐싱합니다. 이후 동일 요청은 원격 호출 없이 메모리에서 즉시 반환하여 대규모 인프라 환경에서도 안정적인 API 서비스를 제공합니다.

 호출 모듈 재조회 설계

재조회 책임을 외부 호출 모듈에 위임하여 엔진 서버 부하를 최소화. 엔진은 단순 Count 반환에 집중.

 서버 COUNT 제한

`web.config`의 Count 값으로 최대 응답 건수를 제어. 인프라 환경에 맞게 튜닝 가능.

🔒 중복·누락 방지

시작시간 + ID 커서 방식으로 페이지 경계에서의
이벤트·태스크 중복·누락을 구조적으로 방지.

< 캐싱 TTL

moREF 키 기반 10초 TTL로 중복 SDK 호출을
차단, 응답 속도 개선 및 vCenter 서버 보호.

C# / .NET 4.5.2

VMware Vim SDK

Memory Cache

Cursor-based Paging

Event · Task 수집

REST API

moREF

web.config

VM 배치 수집 · 보안 통신 · 외부 결재 모듈 연동

Horizon REST API 기반 대규모 VM 상태 수집 배치, C# Launcher-Java 서버 간 RSA+AES 하이브리드 암호화 보안 채널, 그리고 VM 신청(생성/증설/연장)과 사내 결재망(SOAP·REST API) 통합 연동 기능을 담당했다.

C# / .NET

Java

Spring Boot

React

Node.js

Horizon REST API

JPA

MSSQL

RSA + AES

Jenkins

Git

Windows Scheduler

VM 상태 수집 배치 시스템



기존 방식의 한계

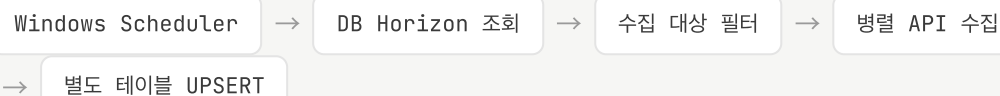
Event DB 폴링 방식으로 실시간 상태 반영 지연. 이벤트 누락 시 상태 불일치 발생. ~25,000대 VM 을 커버하기에 구조적으로 부적합.



개선 방향

Horizon REST API 직접 호출로 전환. 병렬 처리 구조로 수집 시간 최소화. 배치 주기 10 ~ 15분내외 목표 (실측 후 확정).

수집 흐름 요약



STEP 01 - TRIGGER

Windows Task Scheduler 배치 기동

배치 목표 주기는 **10 ~ 15분내외**. API 응답 속도와 DB 처리 시간을 실측한 뒤 최종 확정. 전체 수집 대상 VM은 약 **25,000대**이며, 가장 규모가 큰 단일 호라이즌에 약 12,000대가 수용된다.

STEP 02 - TARGET SELECTION

수집 대상 Horizon 동적 조회 (이중화 제어)

배치 시작 시 DB Horizon 테이블에서 `collect_yn = 'Y'` 인 행만 조회해 수집 대상을 구성한다. 이중화 구성 호라이즌 서버는 Standby를 `'N'` 으로 설정해 중복 수집을 방지하며, DB 컬럼 값 변경만으로 수집 대상을 즉시 전환할 수 있다.

STEP 03 - API STRATEGY

Horizon REST API 조회 전략 결정

포탈 조회 API를 참고하여 **Pool 단위 필터 파라미터 지원 여부**를 사전 확인한다. 필터 지원 시에는 Pool별 분할 병렬 조회로 성능을 최적화하고, 미지원 시에는 호라이즌 단위 전체 일괄 조회 후 메모리 내 처리한다. 현장 담당자에게 **필터 적용 vs 전체 조회 응답 시간 비교 측정을 요청**해 전략을 확정한다.

STEP 04 - PARALLEL COLLECTION

호라이즌별 독립 스레드 병렬 수집

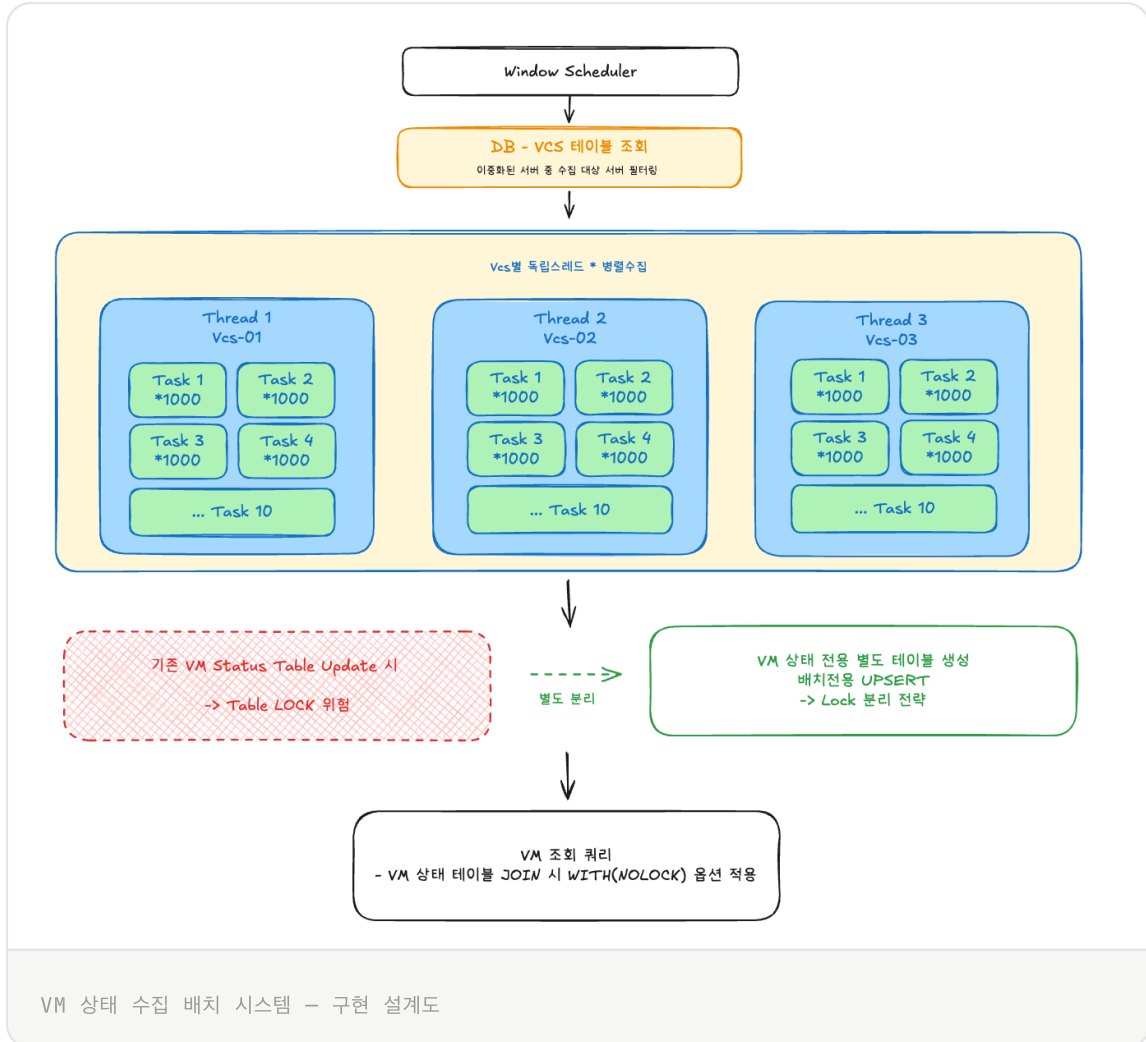
수집 대상 각 Horizon에 독립 스레드를 할당해 병렬로 API를 호출한다. API가 Pool 필터를 지원하면 호라이즌 내에서도 Pool 단위로 서브 태스크를 추가 분할해 병렬화 효과를 극대화한다.

STEP 05 - DB WRITE

별도 VM 상태 테이블에 UPSERT · NOLOCK 적용

수집 결과를 기존 운영 테이블이 아닌 배치 전용 별도 상태 테이블에 UPSERT한다. 10 ~ 15분내외마다 지속 업데이트되므로 기존 테이블에 직접 쓰면 SELECT와의 Lock 경합이 발생한다. 포탈·Trinity의 View, Procedure, Query에서 이 테이블을 JOIN할 때는 `WITH (NOLOCK)` 을 일괄 적용해 Dirty Read를 허용하고 Lock을 우회한다.

설계도



핵심 설계 결정



이중화 호라이즌 수집 제어

Horizon 테이블에 `collect_yn CHAR(1)` 컬럼을 추가. 배치는 'Y' 인 서버만 대상으로 삼아 이중화 구성의 중복 수집을 방지. 운영 중 이중화 전환 시에도 DB 값 변경만으로 즉시 대상을 교체할 수 있어 유연하다.



DB Table Lock 분리

배치 전용 VM 상태 테이블(`vm_status_batch`)을 별도 생성해 Lock을 분리. 포탈·Trinity 조회 프로시저는 해당 테이블 JOIN 시 `WITH (NOLOCK)` 을 적용. 기존 프로시

저의 NOLOCK 패턴을 참고해 동일하게 작성.



수집 대상

~25,000

전체 VM 규모.

최대 단일 호라이즌

~12,000대.



배치 주기

10 ~ 15분내외
목표



처리 방식

병렬 스레드

호라이즌별 독립 스레드.

02

JWT · RSA+AES 하이브리드 보안 통신 모듈

JWT LAUNCH TRIGGER · RSA KEY EXCHANGE · AES-256 ENCRYPTION PROTOCOL

설계 목적

웹 서버가 JWT를 통해 C# Launcher를 안전하게 기동하고, Launcher가 일회용 RSA 키 쌍을 생성해 콜백 요청에 공개키를 포함하여 전송한다. 서버는 JWT를 검증한 뒤 VM 접속 정보를 **AES-256으로 암호화**하고, AES 키는 Launcher의 **RSA 공개키로 Key Wrapping** 하여 응답한다. Launcher는 자신의 RSA 개인키로 AES 키를 복호화한 후 VM 데이터를 복호화해 접속을 수행한다.



Web Server

Java Tomcat



C# Launcher

.NET App

PHASE 1 · LAUNCH TRIGGER

STEP 01

callbackUrl 포함
JWT 생성
(서버 내부)



JWT 생성 (callbackUrl 포함)

payload: { callbackUrl, exp, sub, ... }

STEP 02

.slv 파일 또는



런처 실행 + JWT 전달





JWT — 안전한 런처 기동 및 인증

웹 서버는 `callbackUrl` 을 페이로드에 포함한 JWT를 생성해 런처에 전달한다. 런처가 콜백 요청 시 JWT를 헤더에 포함하면 서버는 서명·만료·인증을 3중 검증한다. 일회성 토큰 구조로 재전송 공격(Replay Attack)을 원천 차단한다.



RSA-OAEP — 일회용 키 교환 (Key Wrapping)

런처는 세션마다 일회용 RSA 키쌍을 생성한다. 공개키를 콜백 요청 헤더에 포함해 전송하면 서버가 이를 사용해 AES 세션키를 RSA-OAEP 방식으로 암호화(Key Wrapping)하여 응답한다. 개인키는 런처만 보유하므로 AES 키는 탈취 불가능하다.



AES-256 — VM 접속 데이터 암호화

실제 VM 접속 정보(호스트·포트·인증 정보)와 프로세스 체크 정보는 AES-256으로 암호화되어 응답 바디에 포함된다. 런처는 복호화한 데이터로 VM 접속 프로그램을 실행하고 커맨드라인으로 명령을 수행해 접속을 완료한다.

보안 설계의 핵심 — JWT로 런처를 안전하게 기동·인증하고, 일회용 RSA 키쌍으로 AES 세션키를 Key Wrapping해 전달한다. VM 접속 데이터는 AES-256으로 암호화하여 네트워크 상에서 평문이 노출되지 않는다. 세션마다 새로운 RSA 키쌍과 AES 키를 생성해 이전 세션 키가 유출되어도 독립적인 보안을 보장한다.

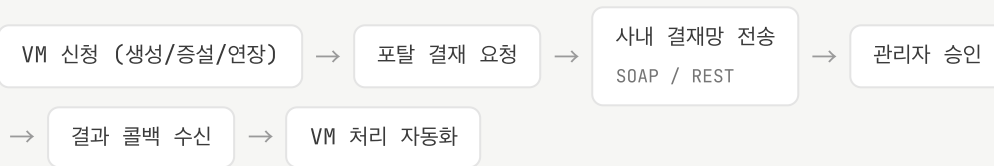
VDI 포탈 외부 통합 결재 모듈 연동

EXTERNAL APPROVAL INTEGRATION · SOAP & REST API

연동 목적

VM 신청(생성 / 증설 / 연장) 기능을 사내 결재망과 연동하여 **관리자 승인 절차 프로세스**를 구축했다. 결재망은 **SOAP**과 **REST API** 두 가지 방식을 병행 지원하며, SOAP 통신을 위한 별도 JAR 모듈을 직접 설계·제작해 포탈에 적용했다.

결재 요청 흐름 요약



주요 구현 내용



VM 신청 결재 연동 (생성 / 증설 / 연장)

VM 생성·증설·연장 신청 시 포탈에서 사내 결재망으로 결재 요청을 자동 전송한다. 관리자가 결재망에서 승인·반려하면 결과가 포탈로 콜백되어, 승인 시 VM 작업이 자동으로 진행되고 반려 시 신청이 취소 처리된다.



REST API 연동

사내 결재망의 REST API 엔드포인트를 통해 결재 요청·조회·콜백 처리를 구현했다. 표준 HTTP 통신 기반으로 포탈 기존 HTTP 클라이언트 모듈을 재사용하여 연동했다.



SOAP 전용 JAR 모듈 개발

사내 결재망 일부 인터페이스가 SOAP 프로토콜만 지원함에 따라, 독립적으로 SOAP 통신이 가능한 **JAR 모듈**을 별도 제작했다. 해당 모듈은 WSDL 기반 SOAP envelope 생성·파싱, 헤더 처리, 응답 매핑 기능을 캡슐화하여 포탈 측에서는 메서드 호출만으로 결재망과 통신할 수 있도록 했다.

SOAP JAR 모듈 분리의 이유 — REST와 달리 SOAP은 WSDL 파싱, envelope 구성, namespace 처리 등 별도의 통신 레이어가 필요하다. 이를 독립 JAR로 캡슐화함으로써 포탈 코

드베이스와 결재망 연동 로직을 명확히 분리하고, 향후 결재망 스펙 변경 시 JAR 모듈만 교체하면 되도록 유지보수성을 확보했다.



신청 유형

3종

VM 생성 / 증설 / 연장.
모든 유형에 결재 프로세스 적용.



연동 방식

SOAP + REST

사내 결재망 프로토콜 병행 지원.
SOAP JAR 모듈 직접 제작.



처리 방식

콜백 자동화

승인 시 VM 작업 자동 진행.
반려 시 신청 자동 취소.

SK하이닉스 SeedVDI 신규 구축 프로젝트 · Dfocus

Batch Collection

Security Module

Approval Integration